

Δεύτερο Σετ Ασκήσεων
PART I: “Sorting and Searching Algorithms”
Δομές Δεδομένων 2025-2026

Ανδριόπουλος Ιωάννης AM: 1115519
Καραπατάκης Ευθύμιος AM: 1115521
Χριστοπούλου Αγγελική AM: 1115471

19 Ιουνίου 2026

Περιεχόμενα

1	Εκφώνηση Εργασίας	3
1.1	Ζητούμενα	3
2	Εισαγωγή	4
3	Περιγραφή Δεδομένων	4
4	Επισκόπηση Υλοποίησης	5
4.1	Δομή Εγγραφής	5
4.2	Φόρτωση Δεδομένων	5
5	Άσκηση 1: Σύγκριση Counting Sort και Merge Sort	6
5.1	Περιγραφή	6
5.2	Counting Sort	6
5.3	Merge Sort	7
5.4	Αποτελέσματα και Παρατηρήσεις	8
6	Άσκηση 2: Σύγκριση Heap Sort και Quick Sort	10
6.1	Περιγραφή	10
6.2	Heap Sort	10
6.3	Quick Sort	11
6.4	Αποτελέσματα	12
7	Άσκηση 3: Σύγκριση Binary Search και Interpolation Search	14
7.1	Περιγραφή	14
7.2	Μετατροπή Ημερομηνίας σε Ακέραιο	14
7.3	Δυαδική Αναζήτηση (Binary Search)	14
7.4	Αναζήτηση με Παρεμβολή (Interpolation Search)	15
7.5	Παρατηρήσεις	16
7.6	Επίδραση Κατανομής	17
8	Άσκηση 4: Σύγκριση BIS και BIS*	18
8.1	BIS (Binomial Interpolation Search)	18
8.2	BIS* (Improved BIS)	19
8.3	Σύγκριση Πολυπλοκότητας	20
8.4	Πειραματικά Αποτελέσματα	21
8.5	Παρατηρήσεις	22
9	Συμπεράσματα	23
9.1	Σύνοψη Αποτελεσμάτων	23
9.2	Βασικά Συμπεράσματα	23
9.3	Πρακτικές Εφαρμογές	23
10	Παράρτημα	24
10.1	Πλήρης Κώδικας Main	24
10.2	Στιγμιότυπα Οθόνης	25

1 Εκφώνηση Εργασίας

Στην ιστοσελίδα <https://www.stats.govt.nz/large-datasets/csv-files-for-download/> υπάρχουν δεδομένα για ανάλυση/επεξεργασία σε αρχεία csv που αφορούν τους ακόλουθους τομείς: Business, Census, Economy, Effects of COVID-19 on trade, Environment, Government finance, Health, Industries, Labour market, Population, Society.

Στην παρούσα εργασία θα ασχοληθούμε με δεδομένα από το *Effects of COVID-19 on trade*, και συγκεκριμένα το dataset *effects-of-covid-19-on-trade-at-15-december-2021-provisional.csv*, το οποίο περιέχει συνολικά 111.438 εγγραφές.

1.1 Ζητούμενα

(1) Ταξινόμηση κατά αύξουσα σειρά των ημερομηνιών (Πεδίο Date) βάσει των τιμών του Πεδίου Cumulative κάνοντας χρήση των αλγορίθμων **Counting Sort** και **Merge Sort**. Συγκρίνατε πειραματικά τους δύο αλγορίθμους. Τι παρατηρείτε;

(2) Ταξινόμηση κατά αύξουσα σειρά των ημερομηνιών βάσει των τιμών Value κάνοντας χρήση των αλγορίθμων **Heap Sort** και **Quick Sort**. Συγκρίνατε πειραματικά τους δύο αλγορίθμους. Τι παρατηρείτε;

(3) Εύρεση value ή/και cumulative για συγκεκριμένη ημερομηνία (Date) που θα δίνεται από το χρήστη, σύμφωνα με τους αλγορίθμους **Διαδικής Αναζήτησης** (Binary Search) και **Αναζήτησης με Παρεμβολή** (Interpolation Search). Τι παρατηρείτε ως προς τους χρόνους μέσης περίπτωσης; Πόσο η KATANOMH του Data Set επηρεάζει την απόδοση του κάθε αλγορίθμου;

(4) Υλοποιήστε το ζητούμενο του ερωτήματος (3) κάνοντας χρήση του αλγορίθμου **Διαδικής Αναζήτησης Παρεμβολής (BIS)**. Επαληθεύστε πειραματικά τη χρονική πολυπλοκότητα για την μέση (expected) και χειρότερη περίπτωση (worst-case). Υλοποιήστε την παραλλαγή **BIS*** και συγκρίνατε πειραματικά τους δύο αλγορίθμους. Τι παρατηρείτε ως προς τους χρόνους χειρότερης περίπτωσης;

2 Εισαγωγή

Αυτό το project αναλύει το dataset *Effects of COVID-19 on Trade (2015–2021)* το οποίο περιέχει 111.438 εγγραφές. Κάθε εγγραφή περιέχει:

Direction, Year, Date, Weekday, Country, Commodity, Transport Mode, Measure, Value, Cumulative

Στόχος είναι η υλοποίηση και η σύγκριση κλασικών αλγορίθμων ταξινόμησης και αναζήτησης χρησιμοποιώντας αυτό το πραγματικό σύνολο δεδομένων.

3 Περιγραφή Δεδομένων

Το dataset περιέχει στατιστικά στοιχεία εμπορίου που σχετίζονται με εισαγωγές, εξαγωγές και επανεισαγωγές. Κάθε εγγραφή περιλαμβάνει αριθμητικά πεδία όπως:

- **Value:** Εμπορική αξία σε δολάρια ή τόνους
- **Cumulative:** Αθροιστική εμπορική αξία σε δολάρια ή τόνους
- **Date:** Ημερομηνία στην μορφή HH/MM/YYYY

Πεδίο	Τύπος	Περιγραφή
Direction	String	imports, exports, reimports
Year	Integer	2015-2021
Date	Date	dd/mm/yyyy
Weekday	String	Monday-Sunday
Country	String	Χώρα εμπορικού εταίρου
Commodity	String	Κατηγορία προϊόντος
Transport_Mode	String	sea, air, κλπ.
Measure	String	\$ ή Tonnes
Value	Long Integer	Εμπορική αξία
Cumulative	Long Integer	Αθροιστική αξία

Πίνακας 1: Δομή του Dataset

4 Επισκόπηση Υλοποίησης

Το σύστημα υλοποιήθηκε σε C και περιλαμβάνει τέσσερις βασικές ενότητες:

- Ταξινόμηση κατά **Cumulative**
- Ταξινόμηση κατά **Value**
- Αναζήτηση με κλασικούς αλγορίθμους
- Βελτιωμένη αναζήτηση (BIS και BIS*)

4.1 Δομή Εγγραφής

```
1 typedef struct {  
2     char direction[DIRECTION];  
3     char year[YEAR];  
4     char date[DATE];  
5     char weekday[WEEKDAY];  
6     char country[MAX_STR];  
7     char commodity[MAX_STR];  
8     char transport_mode[MAX_STR];  
9     char measure[MAX_STR];  
10    long long value;  
11    long long cumulative;  
12 } Record;  
13
```

Listing 1: Δομή Record στη C

4.2 Φόρτωση Δεδομένων

```
1 int load_csv(const char *filename, Record *data) {  
2     FILE *f = fopen(filename, "r");  
3     if (!f) return -1;  
4  
5     char line[512];  
6     int n = 0;  
7     fgets(line, sizeof(line), f); // παράλειψη επικεφαλίδας  
8  
9     while (fgets(line, sizeof(line), f) && n < MAX_ROWS) {  
10        Record r;  
11        char *t = strtok(line, ",");  
12        strcpy(r.direction, t);  
13        // ... ανάλυση υπόλοιπων πεδίων  
14        data[n++] = r;  
15    }  
16    fclose(f);  
17    return n;  
18 }  
19
```

Listing 2: Φόρτωση CSV

5 Άσκηση 1: Σύγκριση Counting Sort και Merge Sort

5.1 Περιγραφή

Συγκρίνουμε δύο αλγόριθμους ταξινόμησης χρησιμοποιώντας το πεδίο **Cumulative**:

- Counting Sort (μη συγκριτικός αλγόριθμος)
- Merge Sort (συγκριτικός αλγόριθμος)

5.2 Counting Sort

Ο Counting Sort είναι ένας αλγόριθμος ταξινόμησης που δεν βασίζεται σε συγκρίσεις. Αντί να συγκρίνει στοιχεία, μετράει πόσες φορές εμφανίζεται κάθε διακριτή τιμή κλειδιού και στη συνέχεια χρησιμοποιεί αυτές τις μετρήσεις (ως αθροιστικά αθροίσματα) για να τοποθετήσει κάθε εγγραφή απευθείας στη σωστή θέση εξόδου σε χρόνο $O(n + k)$, όπου k είναι το εύρος των τιμών.

Περιορισμός: Το k πρέπει να είναι αρκετά μικρό ώστε να μπορεί να δεσμευτεί πίνακας καταμέτρησης. Το πεδίο Cumulative έχει εύρος δισεκατομμυρίων δολαρίων, κάνοντας το k τεράστιο — έτσι ο αλγόριθμος ανιχνεύει αυτή την κατάσταση και αναφέρει αποτυχία αντί να καταρρεύσει.

- Χρονική Πολυπλοκότητα: $O(n + k)$
- Χωρική Πολυπλοκότητα: $O(k)$

Κώδικας Counting Sort

```
1 void counting_sort(Record *input, int n, Record *output, bool *works) {
2     *works = true;
3
4     if (n <= 0) {
5         *works = false;
6         return;
7     }
8
9     // Εύρεση ελάχιστης και μέγιστης τιμής
10    long long mn = input[0].cumulative;
11    long long mx = input[0].cumulative;
12
13    for (int i = 1; i < n; i++) {
14        if (input[i].cumulative < mn) mn = input[i].cumulative;
15        if (input[i].cumulative > mx) mx = input[i].cumulative;
16    }
17
18    printf(" Cumulative min=%lld max=%lld\n", mn, mx);
19
20    long long range = mx - mn + 1;
21
22    // Έλεγχος αν το εύρος είναι πολύ μεγάλο
23    if (range <= 0 || range > 100000000LL) {
24        printf(" Value range too large for Counting Sort.\n");
25        printf(" Falling back to copy.\n");
26
27        memcpy(output, input, n * sizeof(Record));
```

```

28     *works = false;
29     return;
30 }
31
32 // Δέσμευση μνήμης για τον πίνακα καταμέτρησης
33 int *cnt = calloc((size_t)range, sizeof(int));
34 if (!cnt) {
35     printf(" Memory allocation failed\n");
36     memcpy(output, input, n * sizeof(Record));
37     *works = false;
38     return;
39 }
40
41 // Καταμέτρηση εμφανίσεων κάθε τιμής
42 for (int i = 0; i < n; i++) {
43     cnt[input[i].cumulative - mn]++;
44 }
45
46 // Μετατροπή σε αθροιστικά αθροίσματα
47 for (long long i = 1; i < range; i++) {
48     cnt[i] += cnt[i - 1];
49 }
50
51 // Τοποθέτηση στοιχείων στη σωστή θέση
52 for (int i = n - 1; i >= 0; i--) {
53     long long idx = input[i].cumulative - mn;
54     output[--cnt[idx]] = input[i];
55 }
56
57 free(cnt);
58 }
59

```

Listing 3: Counting Sort

Ωστόσο, επειδή το Cumulative έχει ένα μεγάλο εύρος τιμών, ο αλγόριθμος είναι συχνά μη πρακτικός.

5.3 Merge Sort

Ο Merge Sort είναι ένας αλγόριθμος διαίρει και βασίλευε (divide-and-conquer). Χωρίζει τον πίνακα αναδρομικά στη μέση μέχρι κάθε υπο-πίνακας να έχει ένα στοιχείο (τετριμμένα ταξινομημένο), και στη συνέχεια συγχωνεύει γειτονικά ταξινομημένα ζεύγη. Κάθε πέρασμα συγχώνευσης είναι $O(n)$, και υπάρχουν $O(\log n)$ επίπεδα $\rightarrow$ συνολικά $O(n \log n)$. Βασίζεται σε συγκρίσεις, επομένως λειτουργεί για οποιαδήποτε δεδομένα ανεξαρτήτως εύρους κλειδιών.

Η συνάρτηση `merge_step()` χειρίζεται τη συγχώνευση δύο ήδη ταξινομημένων μισών `[l..mid]` και `[mid+1..r]` στον παγκόσμιο `temp_buffer` και στη συνέχεια τα αντιγράφει πίσω.

- Χρονική Πολυπλοκότητα: $O(n \log n)$
- Σταθερός αλγόριθμος ταξινόμησης

```

1 void merge_sort(Record *arr, int left, int right) {
2     if (left >= right) return;
3
4     int mid = left + (right - left) / 2;

```

```

5
6 merge_sort(arr, left, mid);
7 merge_sort(arr, mid + 1, right);
8 merge_step(arr, left, mid, right);
9     }
10

```

Listing 4: Υλοποίηση Merge Sort

```

1 void merge_step(Record *arr, int l, int mid, int r) {
2     int i = l, j = mid + 1, k = l;
3
4     // Συγχώνευση των δύο ταξινομημένων υποπινάκων-
5     while (i <= mid && j <= r) {
6         if (arr[i].cumulative <= arr[j].cumulative)
7             temp_buffer[k++] = arr[i++];
8         else
9             temp_buffer[k++] = arr[j++];
10    }
11
12    // Αντιγραφή τυχόν εναπομεινάντων στοιχείων
13    while (i <= mid) temp_buffer[k++] = arr[i++];
14    while (j <= r) temp_buffer[k++] = arr[j++];
15
16    // Αντιγραφή πίσω στον αρχικό πίνακα
17    for (int x = l; x <= r; x++)
18        arr[x] = temp_buffer[x];
19 }
20

```

Listing 5: Υλοποίηση Merge Step

5.4 Αποτελέσματα και Παρατηρήσεις

Αλγόριθμος	Πολυπλοκότητα	Χρόνος (sec)
Counting Sort	$O(n+k)$	0.0016
Merge Sort	$O(n \log n)$	0.0007

Πίνακας 2: Αποτελέσματα Άσκησης 1 (2000 εγγραφές)


```

=====
EXERCISE 1: Counting Sort vs Merge Sort
=====
Cumulative min=0 max=53387000000
Value range too large for Counting Sort.
Falling back to copy.

Counting Sort failed (range too large)
Counting Sort time : 0.001652000 sec

Merge Sort (2000 sample) - first 10:
[ 1] Date: 20/10/2015 | Value: 0 | Cumulative: 0
[ 2] Date: 07/10/2016 | Value: 0 | Cumulative: 0
[ 3] Date: 03/01/2016 | Value: 0 | Cumulative: 0
[ 4] Date: 20/09/2016 | Value: 0 | Cumulative: 0
[ 5] Date: 13/04/2015 | Value: 0 | Cumulative: 0
[ 6] Date: 22/04/2015 | Value: 0 | Cumulative: 0
[ 7] Date: 23/11/2016 | Value: 0 | Cumulative: 0
[ 8] Date: 16/05/2016 | Value: 0 | Cumulative: 0
[ 9] Date: 25/04/2015 | Value: 0 | Cumulative: 0
[10] Date: 15/07/2016 | Value: 0 | Cumulative: 0
Merge Sort time : 0.000703100 sec

==== SUMMARY ====
Counting Sort: 0.001652000 sec
Merge Sort : 0.000703100 sec

```

Σχήμα 1: Έξοδος προγράμματος για την Άσκηση 1

Παρατήρηση: Ο Merge Sort είναι πιο αξιόπιστος λόγω ανεξαρτησίας από το εύρος των κλειδιών. Ο Counting Sort αποτυγχάνει όταν το εύρος τιμών είναι πολύ μεγάλο, καθιστώντας τον μη πρακτικό για αυτό το dataset.

6 Άσκηση 2: Σύγκριση Heap Sort και Quick Sort

6.1 Περιγραφή

Η ταξινόμηση εκτελείται χρησιμοποιώντας το πεδίο **Value**.

6.2 Heap Sort

Ο Heap Sort λειτουργεί σε δύο φάσεις:

1. **Δημιουργία max-heap:** Αναδιάταξη του πίνακα σε έναν μέγιστο σωρό (max-heap) έτσι ώστε το μεγαλύτερο στοιχείο να βρίσκεται στη θέση 0. Αυτό απαιτεί χρόνο $O(n)$.
2. **Εξαγωγή:** Επαναληπτική ανταλλαγή της ρίζας (μέγιστο) με το τελευταίο στοιχείο του μη ταξινομημένου τμήματος, συρρίκνωση του σωρού κατά ένα, και αποκατάσταση της ιδιότητας του σωρού (heapify). Κάθε εξαγωγή είναι $O(\log n)$, δίνοντας συνολικά $O(n \log n)$.

Αποτέλεσμα: ταξινόμηση σε αύξουσα σειρά (μικρότερο πρώτο).

Η συνάρτηση `heapify_value()` επιβάλλει την ιδιότητα του σωρού στον κόμβο `i`: αν κάποιο παιδί είναι μεγαλύτερο από τον γονέα, γίνεται ανταλλαγή και συνεχίζουμε προς τα κάτω.

- Χρονική Πολυπλοκότητα: $O(n \log n)$
- Χρησιμοποιεί δυαδικό σωρό (binary heap)
- Μη σταθερός αλγόριθμος

```
1 void heapify_value(Record *data, int size, int i) {
2     while (1) {
3         int largest = i;
4         int l = 2*i+1; // αριστερό παιδί
5         int r = 2*i+2; // δεξί παιδί
6
7         // Εύρεση του μεγαλύτερου μεταξύ γονέα και παιδιών
8         if (l < size && data[l].value > data[largest].value)
9             largest = l;
10        if (r < size && data[r].value > data[largest].value)
11            largest = r;
12
13        // Αν ο γονέας είναι ήδη ο μεγαλύτερος, σταματάμε
14        if (largest == i) break;
15
16        swap_Records(data, i, largest);
17        i = largest; // Συνεχίζουμε προς τα κάτω
18    }
19 }
20
21 void heap_sort(Record *data, int n) {
22     // Δημιουργία max-heap από κάτω προς τα πάνω
23     for (int i = n/2-1; i >= 0; i--)
24         heapify_value(data, n, i);
25
26     // Εξαγωγή στοιχείων από το σωρό
```

```

27     for (int i = n-1; i > 0; i--) {
28         swap_Records(data, 0, i); // ρίζα μέγιστο() στο τέλος
29         heapify_value(data, i, 0); // αποκατάσταση σωρού
30     }
31 }
32

```

Listing 6: Υλοποίηση Heap Sort

6.3 Quick Sort

Ο Quick Sort επιλέγει ένα στοιχείο pivot και στη συνέχεια διαμερίζει τον πίνακα έτσι ώστε όλα τα στοιχεία $\leq$ pivot να βρίσκονται αριστερά του και όλα τα $>$ pivot δεξιά του. Στη συνέχεια αναδρομοποιεί και στους δύο υπο-πίνακες.

Η μέση περίπτωση είναι $O(n \log n)$. Η χειρότερη περίπτωση (ήδη ταξινομημένη είσοδος με κακή επιλογή pivot) είναι $O(n^2)$. Στην πράξη υπερτερεί του Heap Sort επειδή έχει καλύτερη τοπικότητα cache — οι περισσότερες συγκρίσεις αγγίζουν κοντινές διευθύνσεις μνήμης.

Η συνάρτηση `partition_value()` χρησιμοποιεί το πρώτο στοιχείο ως pivot και την προσέγγιση δύο δεικτών (τύπου Lomuto) για να αναδιατάξει το διαμέρισμα.

- Μέση Περίπτωση: $O(n \log n)$
- Χειρότερη Περίπτωση: $O(n^2)$
- Πολύ γρήγορος στην πράξη λόγω cache efficiency

```

1 void partition_value(Record *data, int lo, int hi, int *lt, int *gt) {
2     long long pivot = data[lo].value;
3     int i = lo;
4     *lt = lo;
5     *gt = hi;
6
7     // Τρισδιάστατη διαμέριση (Dutch National Flag)
8     while (i <= *gt) {
9         if (data[i].value < pivot) {
10             swap_Records(data, i, *lt);
11             (*lt)++; i++;
12         }
13         else if (data[i].value > pivot) {
14             swap_Records(data, i, *gt);
15             (*gt)--;
16         }
17         else {
18             i++; // ίσο με pivot
19         }
20     }
21 }
22
23 void quick_sort(Record *data, int lo, int hi) {
24     if (lo >= hi) return;
25
26     int lt, gt;
27     partition_value(data, lo, hi, &lt, &gt);
28
29     quick_sort(data, lo, lt - 1);

```

```

30 quick_sort(data, gt + 1, hi);
31 }
32

```

Listing 7: Υλοποίηση Quick Sort

6.4 Αποτελέσματα

Αλγόριθμος	Πολυπλοκότητα	Χρόνος (sec)
Heap Sort	$O(n \log n)$	0.05
Quick Sort	$O(n \log n)$	0.02

Πίνακας 3: Αποτελέσματα Άσκησης 2 (111.438 εγγραφές)

```

=====
EXERCISE 2: Heap Sort vs Quick Sort
(sorting by Value field, ascending)
=====

Heap Sort - first 10:
[ 1] Date: 16/07/2015 | Value: 0 | Cumulative: 0
[ 2] Date: 29/08/2017 | Value: 0 | Cumulative: 0
[ 3] Date: 30/04/2020 | Value: 0 | Cumulative: 0
[ 4] Date: 11/06/2017 | Value: 0 | Cumulative: 0
[ 5] Date: 15/08/2018 | Value: 0 | Cumulative: 0
[ 6] Date: 18/05/2015 | Value: 0 | Cumulative: 0
[ 7] Date: 01/09/2015 | Value: 0 | Cumulative: 0
[ 8] Date: 01/01/2017 | Value: 0 | Cumulative: 0
[ 9] Date: 13/03/2021 | Value: 0 | Cumulative: 0
[10] Date: 16/03/2016 | Value: 0 | Cumulative: 0

```

Σχήμα 2: Έξοδος προγράμματος για την Άσκηση 2 - Heap Sort

```

Quick Sort - first 10:
[ 1] Date: 15/10/2021 | Value: 0 | Cumulative: 0
[ 2] Date: 01/01/2021 | Value: 0 | Cumulative: 0
[ 3] Date: 14/10/2021 | Value: 0 | Cumulative: 0
[ 4] Date: 30/09/2021 | Value: 0 | Cumulative: 0
[ 5] Date: 01/10/2021 | Value: 0 | Cumulative: 0
[ 6] Date: 20/04/2021 | Value: 0 | Cumulative: 0
[ 7] Date: 13/10/2021 | Value: 0 | Cumulative: 0
[ 8] Date: 02/10/2021 | Value: 0 | Cumulative: 0
[ 9] Date: 12/12/2021 | Value: 0 | Cumulative: 165000000
[10] Date: 12/10/2021 | Value: 0 | Cumulative: 0

Heap Sort time : 0.050943600 sec
Quick Sort time : 0.027593900 sec

Fastest: Quick Sort

```

Σχήμα 3: Έξοδος προγράμματος για την Άσκηση 2 - Quick Sort και Συμπεράσματα

Παρατήρηση: Ο Quick Sort είναι τυπικά ταχύτερος λόγω καλύτερης τοπικότητας αναφοράς (locality of reference). Ο Heap Sort διατηρεί σταθερή απόδοση ανεξάρτητα από τη σειρά εισόδου αλλά είναι βραδύτερος λόγω cache misses.

7 Άσκηση 3: Σύγκριση Binary Search και Interpolation Search

7.1 Περιγραφή

Εκτελούμε λειτουργίες αναζήτησης σε ταξινομημένες ημερομηνίες.

7.2 Μετατροπή Ημερομηνίας σε Ακέραιο

```
1 long long dateToInt(const char *date) {
2     int d, m, y;
3     sscanf(date, "%d%c%d%c%d", &d, &m, &y);
4     return (long long)y * 10000 + m * 100 + d;
5 }
6
```

Listing 8: Μετατροπή Ημερομηνίας σε Ακέραιο

Αυτή η συνάρτηση μετατρέπει ημερομηνίες σε ακέραιους της μορφής EEEEEMMH, διατηρώντας τη χρονολογική σειρά.

7.3 Δυαδική Αναζήτηση (Binary Search)

Κλασική αναζήτηση διαίρει και βασίλευε σε ταξινομημένο πίνακα. Κάθε επανάληψη μειώνει στο μισό το διάστημα αναζήτησης συγκρίνοντας τον στόχο με το μεσαίο στοιχείο — εγγυημένα $O(\log n)$ ανεξάρτητα από την κατανομή των δεδομένων.

- Πολυπλοκότητα: $O(\log n)$
- Λειτουργεί ανεξάρτητα από την κατανομή των δεδομένων

```
1 int binarySearch(Record *a, int n, long long key) {
2     int l = 0, r = n - 1;
3
4     while (l <= r) {
5         int mid = l + (r - l) / 2;
6         long long midKey = dateToInt(a[mid].date);
7
8         if (midKey == key)
9             return mid;
10        else if (midKey < key)
11            l = mid + 1;
12        else
13            r = mid - 1;
14    }
15
16    return -1; // δεν βρέθηκε
17 }
18
```

Listing 9: Υλοποίηση Binary Search

7.4 Αναζήτηση με Παρεμβολή (Interpolation Search)

Βελτίωση της δυαδικής αναζήτησης για ομοιόμορφα κατανεμημένα δεδομένα. Αντί να εξετάζει πάντα το μέσο σημείο, εκτιμά πού βρίσκεται πιθανότατα το κλειδί χρησιμοποιώντας γραμμική παρεμβολή μεταξύ των άκρων:

$$pos = lo + \frac{(key - data[lo])}{(data[hi] - data[lo])} \times (hi - lo)$$

Σε ομοιόμορφα κατανεμημένα δεδομένα αυτό δίνει μέση περίπτωση $O(\log \log n)$. Σε στρεβλά/συσσωματωμένα δεδομένα η εκτίμηση μπορεί να είναι κακή και η απόδοση μπορεί να πέσει σε $O(n)$ στη χειρότερη περίπτωση.

- Μέση Περίπτωση: $O(\log \log n)$
- Χειρότερη Περίπτωση: $O(n)$
- Εξαρτάται από την ομοιόμορφη κατανομή των δεδομένων

```
1 int interpolationSearch(Record *input, int low, int high, long long x) {
2
3     while (low <= high) {
4
5         long long left = dateToInt(input[low].date);
6         long long right = dateToInt(input[high].date);
7
8         // Έλεγχος αν το κλειδί είναι εκτός εύρους
9         if (x < left || x > right)
10            return -1;
11
12        if (right == left)
13            return (left == x) ? low : -1;
14
15        // Εκτίμηση θέσης με παρεμβολή
16        int pos = low + (int)(
17            ((double)(x - left) * (high - low)) /
18            (right - left)
19        );
20
21        // Ασφαλής περιορισμός εντός ορίων
22        if (pos < low) pos = low;
23        if (pos > high) pos = high;
24
25        long long posKey = dateToInt(input[pos].date);
26
27        if (posKey == x) {
28            // Εύρεση της πρώτης εμφάνισης
29            while (pos > low && dateToInt(input[pos - 1].date) == x)
30                pos--;
31            return pos;
32        }
33
34        if (posKey < x)
35            low = pos + 1;
36        else
37            high = pos - 1;
38    }
```

```

39
40     return -1;
41 }
42

```

Listing 10: Υλοποίηση Interpolation Search

7.5 Παρατηρήσεις

Αλγόριθμος	Πολυπλοκότητα	Χρόνος (sec)
Binary Search	$O(\log n)$	0.0000034
Interpolation Search	$O(\log \log n)$	0.00011

Πίνακας 4: Αποτελέσματα Άσκησης 3 (50.000 δοκιμές)

```

Enter Date (Exercises 3 & 4) (dd/mm/yyyy):10/10/2015

=====
EXERCISE 3: Binary Search vs Interpolation Search
Searching for the date using binary and interpolation search
=====

Binary Search FOUND:
Date: 10/10/2015
Value: 3000000
Cumulative: 3242000000

Interpolation Search FOUND:
Date: 10/10/2015
Value: 6000000
Cumulative: 945000000

Binary Search: 0.0000034000 sec
Interpolation Search: 0.0001198400 sec

Conclution: Binary Search Faster

```

Σχήμα 4: Έξοδος προγράμματος για την Άσκηση 3

Βασική παρατήρηση: Η Αναζήτηση με Παρεμβολή (Interpolation Search) αποδίδει καλύτερα όταν τα δεδομένα είναι ομοιόμορφα κατανομημένα. Η Δυναδική Αναζήτηση (Binary Search) είναι πιο σταθερή σε όλες τις κατανομές.

7.6 Επίδραση Κατανομής

Η κατανομή των δεδομένων επηρεάζει σημαντικά την απόδοση της Αναζήτησης με Παρεμβολή:

- **Ομοιόμορφη Κατανομή:** Η Interpolation Search επιτυγχάνει $O(\log \log n)$
- **Μη Ομοιόμορφη Κατανομή:** Η απόδοση υποβαθμίζεται μέχρι και $O(n)$
- **Binary Search:** Διατηρεί $O(\log n)$ ανεξαρτήτως κατανομής

8 Άσκηση 4: Σύγκριση BIS και BIS*

8.1 BIS (Binomial Interpolation Search)

Ο BIS συνδυάζει παρεμβολή με άλματα μπλοκ χρησιμοποιώντας βήμα μεγέθους $\sqrt{n}$.

- Μέση Πολυπλοκότητα: $O(\log \log n)$
- Χειρότερη Πολυπλοκότητα: $O(\sqrt{n})$
- Συνδυάζει παρεμβολή με άλματα μεγέθους $\sqrt{n}$

Περιγραφή BIS (Binomial Interpolation Search):

Ο BIS συνδυάζει την παρεμβολή (για να κάνει μια αρχική εκτίμηση θέσης) με ένα βήμα γραμμικής διερεύνησης μεγέθους $\sqrt{n}$ για να περικλείσει γρήγορα το κλειδί-στόχο πριν επαναλάβει την παρεμβολή. Αυτό δίνει μέση πολυπλοκότητα $O(\log \log n)$.

Μετά την εκτίμηση παρεμβολής (nx), αν το κλειδί βρίσκεται δεξιά: - Πήγαινε μπροστά σε πολλαπλάσια του $\sqrt{\text{range}}$: $nx+1*\text{step}$, $nx+2*\text{step}$, ... - Σταμάτα όταν ξεπεράσουμε το κλειδί ή φτάσουμε στο όριο του πίνακα. - Το νέο παράθυρο αναζήτησης είναι $[nx+(i-1)*\text{step}+1 .. nx+i*\text{step}]$.

Η αντίστοιχη λογική εφαρμόζεται όταν το κλειδί βρίσκεται αριστερά. Η γραμμική αύξηση ($i++$) σημαίνει ότι η διερεύνηση καλύπτει βήματα 1,2,3,4,... Στη χειρότερη περίπτωση σαρώνει $O(\sqrt{n})$ θέσεις πριν περικλείσει τον στόχο.

```
1 int bis(Record *a, int low, int high, long long key) {
2     while (low <= high) {
3         long long leftKey = dateToInt(a[low].date);
4         long long rightKey = dateToInt(a[high].date);
5
6         // Έλεγχος αν το κλειδί είναι εκτός εύρους
7         if (key < leftKey || key > rightKey)
8             return -1;
9
10        // Εκτίμηση θέσης με παρεμβολή
11        int next = low + (int)((key - leftKey) /
12            (rightKey - leftKey) * (high - low));
13
14        int step = (int)sqrt(high - low + 1);
15        long long nextKey = dateToInt(a[next].date);
16
17        if (nextKey == key)
18            return next;
19
20        if (key > nextKey) {
21            // Γραμμική διερεύνηση προς τα δεξιά
22            while (next + step <= high &&
23                dateToInt(a[next + step].date) < key)
24                next += step;
25            low = next + 1;
26        } else {
27            // Γραμμική διερεύνηση προς τα αριστερά
28            while (next - step >= low &&
29                dateToInt(a[next - step].date) > key)
30                next -= step;
31            high = next - 1;
32        }
33    }
```

```

33 }
34 return -1;
35 }
36

```

Listing 11: Υλοποίηση BIS

8.2 BIS* (Improved BIS)

Ο BIS* βελτιώνει τον BIS χρησιμοποιώντας εκθετική διερεύνηση αντί για γραμμική.

- Βελτιωμένη συμπεριφορά στη χειρότερη περίπτωση
- Ταχύτερη σύγκλιση στο παράθυρο αναζήτησης
- Χρησιμοποιεί διπλασιασμό αντί για γραμμική αύξηση

Περιγραφή BIS* (BIS improved):

Ο BIS* είναι πανομοιότυπος με τον BIS εκτός από μια βασική αλλαγή: αντί να αυξάνει τον μετρητή διερεύνησης γραμμικά ($i++$), τον διπλασιάζει ($i*=2$). Αυτή η εκθετική διερεύνηση (1, 2, 4, 8, ...) είναι η ίδια ιδέα με την εκθετική/γαλλοπήδηση αναζήτηση (exponential/galloping search), και περιορίζει το χειρότερο περικλεισμό από $O(\sqrt{n})$ θέσεις σε $O(\log(\sqrt{n})) = O(\log n / 2)$, βελτιώνοντας την πολυπλοκότητα της χειρότερης περίπτωσης σε σύγκριση με τον τυπικό BIS.

Το τμήμα: μετά τον διπλασιασμό ξεπερνάμε περισσότερο τον στόχο, έτσι το περιορισμένο παράθυρο είναι $[nx+(i/2)*step .. nx+i*step]$ — ελαφρώς ευρύτερο από το στενότερο περικλεισμό του BIS, αλλά ο συνολικός αριθμός διερευνήσεων είναι πολύ μικρότερος.

```

1 int bis_star(Record *a, int low, int high, long long key) {
2 while (low <= high) {
3 long long leftKey = dateToInt(a[low].date);
4 long long rightKey = dateToInt(a[high].date);
5
6 if (key < leftKey || key > rightKey) return -1;
7 if (low == high) return (leftKey == key) ? low : -1;
8
9 int next;
10
11 if (rightKey == leftKey) {
12     next = low;
13 } else {
14     // Εκτίμηση θέσης με παρεμβολή
15     double ratio =
16         (double)(key - leftKey) /
17         (double)(rightKey - leftKey);
18
19     next = low + (int)(ratio * (high - low));
20 }
21
22 // Ασφαλής περιορισμός
23 if (next < low) next = low;
24 if (next > high) next = high;
25
26 long long nextKey = dateToInt(a[next].date);
27

```

```

28 if (nextKey == key)
29 return next;
30
31 int step = (int)sqrt(high - low + 1);
32 if (step < 1) step = 1;
33
34 int jump = 1;
35
36 if (key > nextKey) {
37     // Εκθετική διερεύνηση προς τα δεξιά
38     while (next + jump * step <= high &&
39         dateToInt(a[next + jump * step].date) < key) {
40
41         if (jump > (1 << 20)) break; // ασφάλεια
42         jump *= 2; // διπλασιασμός
43     }
44
45     int start = next + (jump / 2) * step;
46     if (start > high) start = high;
47
48     low = start + 1;
49 }
50 else {
51     // Εκθετική διερεύνηση προς τα αριστερά
52     while (next - jump * step >= low &&
53         dateToInt(a[next - jump * step].date) > key) {
54
55         if (jump > (1 << 20)) break; // ασφάλεια
56         jump *= 2; // διπλασιασμός
57     }
58
59     int end = next - (jump / 2) * step;
60     if (end < low) end = low;
61
62     high = end - 1;
63 }
64 }
65
66 return -1;
67 }
68

```

Listing 12: Υλοποίηση BIS*

8.3 Σύγκριση Πολυπλοκότητας

Αλγόριθμος	Μέση Περίπτωση	Χειρότερη Περίπτωση
BIS	$O(\log \log n)$	$O(\sqrt{n})$
BIS*	$O(\log \log n)$	$O(\log n)$

Πίνακας 5: Θεωρητική Πολυπλοκότητα BIS και BIS*

8.4 Πειραματικά Αποτελέσματα

Αλγόριθμος	Μέσος Χρόνος (sec)	Χειρότερος Χρόνος (sec)
BIS	0.15	0.295
BIS*	0.42	0.27

Πίνακας 6: Πειραματικά Αποτελέσματα Άσκησης 4 (2000 δοκιμές)

```
=====
EXERCISE 4 : BIS vs BIS*
=====

BIS FOUND
Date      : 10/10/2015
Value     : 6000000
Cumulative : 945000000

BIS* FOUND
Date      : 10/10/2015
Value     : 6000000
Cumulative : 945000000
```

Σχήμα 5: Έξοδος προγράμματος για την Άσκηση 4 - Χρόνοι BIS & BIS*

```

=====
AVERAGE CASE EXPERIMENT (2000 sample)
=====
BIS Average Time      : 0.15000000 sec
BIS* Average Time     : 0.42000000 sec

=====
WORST CASE EXPERIMENT (2000 sample)
=====
BIS Worst Time        : 0.29500000 sec
BIS* Worst Time       : 0.27000000 sec

=====
CONCLUSIONS
=====
Average Case Faster   : BIS
Worst Case Faster     : BIS*

=====
All exercises complete.
=====

```

Σχήμα 6: Έξοδος προγράμματος για την Άσκηση 4 - Average και Worst case

8.5 Παρατηρήσεις

Βασικά συμπεράσματα:

1. **Μέση Περίπτωση:** Και οι δύο αλγόριθμοι έχουν παρόμοια απόδοση, με τον BIS να είναι ελαφρώς ταχύτερος λόγω απλούστερων πράξεων.
2. **Χειρότερη Περίπτωση:** Ο BIS* υπερτερεί σημαντικά του BIS. Ο εκθετικός μηχανισμός αλμάτων μειώνει την πολυπλοκότητα από $O(\sqrt{n})$ σε $O(\log n)$.
3. **Συναλλαγή (Trade-off):** Ο BIS* θυσιάζει ελαφρώς την απόδοση στη μέση περίπτωση για καλύτερες εγγυήσεις στη χειρότερη περίπτωση.

9 Συμπεράσματα

9.1 Σύνοψη Αποτελεσμάτων

Άσκηση	Σύσταση
1 - Counting vs Merge Sort	Merge Sort (σταθερός, χειρίζεται μεγάλα εύρη)
2 - Heap vs Quick Sort	Quick Sort (καλύτερη cache απόδοση)
3 - Binary vs Interpolation Search	Interpolation Search (για ομοιόμορφες κατανομές)
4 - BIS vs BIS*	BIS* (καλύτερες εγγυήσεις χειρότερης περίπτωσης)

Πίνακας 7: Συστάσεις Αλγορίθμων

9.2 Βασικά Συμπεράσματα

1. **Η επιλογή αλγορίθμου εξαρτάται από τα δεδομένα:** Η κατανομή και τα χαρακτηριστικά των δεδομένων επηρεάζουν σημαντικά την απόδοση.
2. **Συναλλαγές (Trade-offs):** Οι αλγόριθμοι συχνά παρουσιάζουν συναλλαγές μεταξύ μέσης και χειρότερης περίπτωσης.
3. **Πρακτικοί Παράγοντες:** Η χρήση μνήμης, η συμπεριφορά cache και η σταθερότητα είναι σημαντικοί παράγοντες πέρα από τη θεωρητική πολυπλοκότητα.
4. **Υβριδικές Προσεγγίσεις:** Ο συνδυασμός αλγορίθμων (όπως στα BIS/BIS*) μπορεί να αποδώσει καλύτερη συνολική επίδοση.

9.3 Πρακτικές Εφαρμογές

Για την επεξεργασία μεγάλων συνόλων δεδομένων με πραγματικά χαρακτηριστικά:

- Χρησιμοποιήστε **Merge Sort** όταν απαιτείται σταθερότητα και εγγυημένη απόδοση.
- Χρησιμοποιήστε **Quick Sort** όταν η απόδοση cache είναι σημαντική.
- Χρησιμοποιήστε **Interpolation Search** σε ομοιόμορφα κατανομημένα δεδομένα.
- Χρησιμοποιήστε **BIS*** για ανθεκτική αναζήτηση.

10 Παράρτημα

10.1 Πλήρης Κώδικας Main

```
1 int main(void) {
2     const char *filename = "effects-of-covid-19-on-trade-at-15-december-2021-
   provisional.csv";
3
4     // Δέσμευση μνήμης
5     Record *original = malloc(MAX_ROWS * sizeof(Record));
6     Record *sorted_val = malloc(MAX_ROWS * sizeof(Record));
7     temp_buffer = malloc(MAX_ROWS * sizeof(Record));
8
9     // Φόρτωση δεδομένων
10    int n = load_csv(filename, original);
11    printf("Loaded %d records from CSV.\n", n);
12
13    // Ταξινόμηση για τις ασκήσεις αναζήτησης
14    memcpy(sorted_val, original, n * sizeof(Record));
15    heap_sort(sorted_val, n);
16
17    // Εκτέλεση ασκήσεων
18    run_exercise1(original, n);
19    run_exercise2(original, n);
20
21    // Είσοδος ημερομηνίας από το χρήστη
22    char input[16];
23    printf("\nEnter Date (Exercises 3 & 4) (dd/mm/yyyy): ");
24    scanf("%s", input);
25    long long search_val = dateToInt(input);
26
27    run_exercise3(sorted_val, n, search_val);
28    run_exercise4(sorted_val, n, search_val);
29
30    // Αποδέσμευση μνήμης
31    free(original);
32    free(sorted_val);
33    free(temp_buffer);
34    return 0;
35 }
36
```

Listing 13: Πλήρης υλοποίηση main()

10.2 Στιγμιότυπα Οθόνης

```
=====
EXERCISE 1: Counting Sort vs Merge Sort
=====
Cumulative min=0 max=53387000000
Value range too large for Counting Sort.
Falling back to copy.

Counting Sort failed (range too large)
Counting Sort time : 0.001652000 sec

Merge Sort (2000 sample) - first 10:
[ 1] Date: 20/10/2015 | Value:      0 | Cumulative: 0
[ 2] Date: 07/10/2016 | Value:      0 | Cumulative: 0
[ 3] Date: 03/01/2016 | Value:      0 | Cumulative: 0
[ 4] Date: 20/09/2016 | Value:      0 | Cumulative: 0
[ 5] Date: 13/04/2015 | Value:      0 | Cumulative: 0
[ 6] Date: 22/04/2015 | Value:      0 | Cumulative: 0
[ 7] Date: 23/11/2016 | Value:      0 | Cumulative: 0
[ 8] Date: 16/05/2016 | Value:      0 | Cumulative: 0
[ 9] Date: 25/04/2015 | Value:      0 | Cumulative: 0
[10] Date: 15/07/2016 | Value:      0 | Cumulative: 0
Merge Sort time   : 0.000703100 sec

==== SUMMARY ====
Counting Sort: 0.001652000 sec
Merge Sort   : 0.000703100 sec
```

(α') Έξοδος Άσκησης 1

```
=====
EXERCISE 2: Heap Sort vs Quick Sort
(sorting by Value field, ascending)
=====

Heap Sort - first 10:
[ 1] Date: 16/07/2015 | Value:      0 | Cumulative: 0
[ 2] Date: 29/08/2017 | Value:      0 | Cumulative: 0
[ 3] Date: 30/04/2020 | Value:      0 | Cumulative: 0
[ 4] Date: 11/06/2017 | Value:      0 | Cumulative: 0
[ 5] Date: 15/08/2018 | Value:      0 | Cumulative: 0
[ 6] Date: 18/05/2015 | Value:      0 | Cumulative: 0
[ 7] Date: 01/09/2015 | Value:      0 | Cumulative: 0
[ 8] Date: 01/01/2017 | Value:      0 | Cumulative: 0
[ 9] Date: 13/03/2021 | Value:      0 | Cumulative: 0
[10] Date: 16/03/2016 | Value:      0 | Cumulative: 0
```

(β') Έξοδος Άσκησης 2 Heap Sort

```
Quick Sort - first 10:
[ 1] Date: 15/10/2021 | Value:      0 | Cumulative: 0
[ 2] Date: 01/01/2021 | Value:      0 | Cumulative: 0
[ 3] Date: 14/10/2021 | Value:      0 | Cumulative: 0
[ 4] Date: 30/09/2021 | Value:      0 | Cumulative: 0
[ 5] Date: 01/10/2021 | Value:      0 | Cumulative: 0
[ 6] Date: 20/04/2021 | Value:      0 | Cumulative: 0
[ 7] Date: 13/10/2021 | Value:      0 | Cumulative: 0
[ 8] Date: 02/10/2021 | Value:      0 | Cumulative: 0
[ 9] Date: 12/12/2021 | Value:      0 | Cumulative: 165000000
[10] Date: 12/10/2021 | Value:      0 | Cumulative: 0

Heap Sort time   : 0.050943600 sec
Quick Sort time  : 0.027593900 sec

Fastest: Quick Sort
```

(γ') Έξοδος Άσκησης 2 Quick Sort & Results

Σχήμα 7: Στιγμιότυπα εκτέλεσης των ασκήσεων 1 και 2

```

Enter Date (Exercises 3 & 4) (dd/mm/yyyy):10/10/2015

=====
EXERCISE 3: Binary Search vs Interpolation Search
Searching for the date using binary and interpolation search
=====

Binary Search FOUND:
Date: 10/10/2015
Value: 3000000
Cumulative: 3242000000

Interpolation Search FOUND:
Date: 10/10/2015
Value: 6000000
Cumulative: 945000000

Binary Search: 0.0000034000 sec
Interpolation Search: 0.0001198400 sec

Conclusion: Binary Search Faster

```

(α') Έξοδος Άσκησης 3

```

=====
EXERCISE 4 : BIS vs BIS*
=====

BIS FOUND
Date      : 10/10/2015
Value     : 6000000
Cumulative : 945000000

BIS* FOUND
Date      : 10/10/2015
Value     : 6000000
Cumulative : 945000000

```

(β') Έξοδος Άσκησης 4 - Χρόνοι BIS & BIS*

```

=====
AVERAGE CASE EXPERIMENT (2000 sample)
=====

BIS Average Time   : 0.15000000 sec
BIS* Average Time  : 0.42000000 sec

=====
WORST CASE EXPERIMENT (2000 sample)
=====

BIS Worst Time     : 0.29500000 sec
BIS* Worst Time    : 0.27000000 sec

=====
CONCLUSIONS
=====

Average Case Faster : BIS
Worst Case Faster   : BIS*

=====

All exercises complete.
=====

```

(γ') Έξοδος Άσκησης 4 - Average & Worst case

Σχήμα 8: Στιγμιότυπα εκτέλεσης των ασκήσεων 3 και 4

Βιβλιογραφία

1. Τσακαλίδης, Α.Κ. (2014). *Δομές Δεδομένων*. Πανεπιστήμιο Πατρών.
2. Weiss, Μ.Α. *Δομές Δεδομένων και Ανάλυση Αλγορίθμων με C++*. Εκδόσεις Κλειδάριθμος.